Redes profundas y regularización

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Recap
2. Teorema de aproximación universal
3. Redes profundas
4. Regularización
5. Cierre

Recap: Single Layer Neural Networks

- Una red de una capa oculta con K unidades:

$$f(X) = f^{(output)}\left[\beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j\right)\right]$$

- $g(\cdot)$ no lineal: ahí está "la magia".
- Se entrena con backpropagation + gradiente descendente (SGD, Adam).
- Hoy, cuatro preguntas:
  1. ¿Qué tan flexible es este modelo?
  2. Si una capa alcanza, ¿para qué apilar varias?
  3. ¿Cómo evitamos que memorice los datos?
  4. Clasificación multiclase: MNIST.

Agenda

1. Recap
2. Teorema de aproximación universal
3. Redes profundas
4. Regularización
5. Cierre

Teorema de aproximación universal

- Sea g una activación **no polinomial** (sigmoide, tanh, ReLU, ...).
- Para toda función continua f en un compacto $\mathcal{K} \subset \mathbb{R}^p$ y todo $\epsilon > 0$, existen K y parámetros (β, w) tales que

$$\sup_{x \in \mathcal{K}} \left| f(x) - \hat{f}(x) \right| < \epsilon, \qquad \hat{f}(x) = \beta_0 + \sum_{k=1}^{K} \beta_k \, g\left(w_{k0} + w_k' x\right)$$

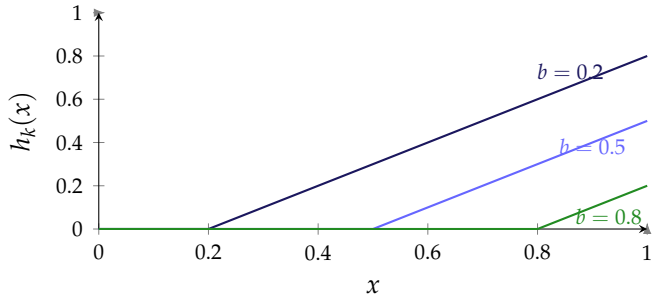
- Una **sola capa oculta** basta... si es suficientemente ancha.
- Cybenko (1989) para sigmoides; Hornik, Stinchcombe & White (1989); Leshno, Lin, Pinkus & Schocken (1993): cualquier activación no polinomial, incluida ReLU.

¿Por qué funciona? Una ReLU es un quiebre

- En una dimensión, tomemos $w_k = 1$ y $w_{k0} = -b_k$:

$$h_k(x) = \max(0,\, x - b_k)$$

- Vale cero hasta b_k, y después sube con pendiente 1.
- Cada unidad oculta aporta **un quiebre** en el punto b_k.



¿Por qué funciona? Sumar quiebres

- La red suma K quiebres con pesos β_k, ordenando $b_1 < b_2 < \cdots < b_K$:

$$\hat{f}(x) = \beta_0 + \sum_{k=1}^{K} \beta_k \max(0,\, x - b_k)$$

- $\hat{f}$ es **lineal por tramos**, con quiebres en los b_k.

¿Por qué funciona? Sumar quiebres

- La red suma K quiebres con pesos β_k, ordenando $b_1 < b_2 < \cdots < b_K$:

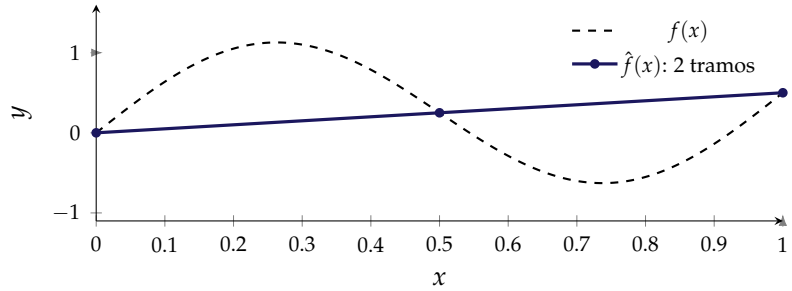
$$\hat{f}(x) = \beta_0 + \sum_{k=1}^{K} \beta_k \max(0,\, x - b_k)$$

- $\hat{f}$ es **lineal por tramos**, con quiebres en los b_k.
- En el tramo (b_k, b_{k+1}) están "prendidas" las unidades $1, \ldots, k$, así que la pendiente es

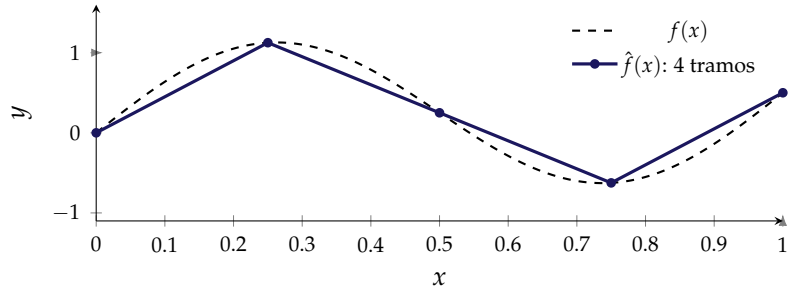
$$s_k = \sum_{j=1}^{k} \beta_j \qquad \Longrightarrow \qquad \beta_k = s_k - s_{k-1}$$

- β_k es el **cambio de pendiente** en b_k. Para pasar por los puntos $(b_k, f(b_k))$ basta con elegir s_k igual a la pendiente de la secante entre b_k y b_{k+1}.

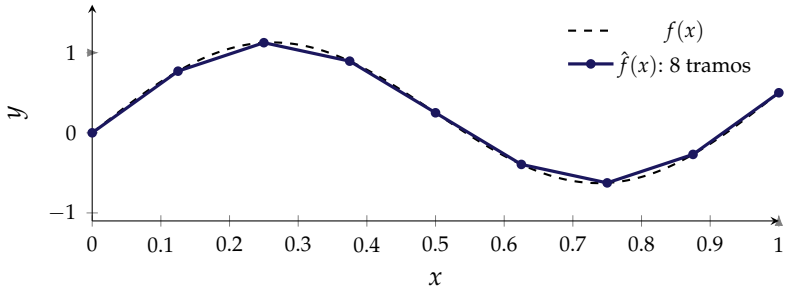
Aproximando f con quiebres



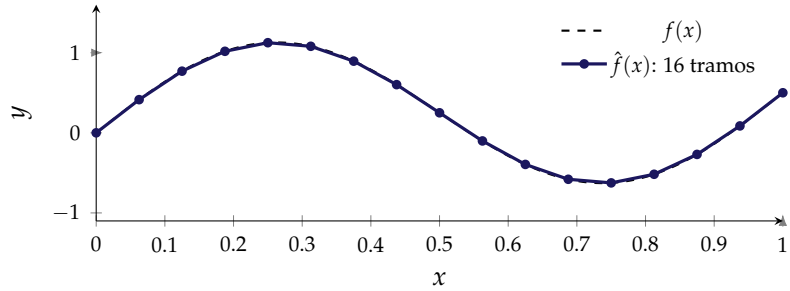
Aproximando f con quiebres



Aproximando f con quiebres



Aproximando f con quiebres



El argumento, completo (en una dimensión)

- Partimos $[0,1]$ en K tramos de ancho $1/K$ y conectamos los puntos $(x_k, f(x_k))$: eso usa del orden de K unidades ReLU.
- Dentro de cada tramo, $\hat{f}(x)$ es un promedio ponderado de $f(x_k)$ y $f(x_{k+1})$, así que

$$\sup_x |f(x) - \hat{f}(x)| \;\leq\; \omega_f\left(\tfrac{1}{K}\right), \qquad \omega_f(\delta) = \sup_{|x-x'|\leq\delta} |f(x) - f(x')|$$

- f continua en un compacto es uniformemente continua: $\omega_f(1/K) \to 0$ cuando $K \to \infty$. □

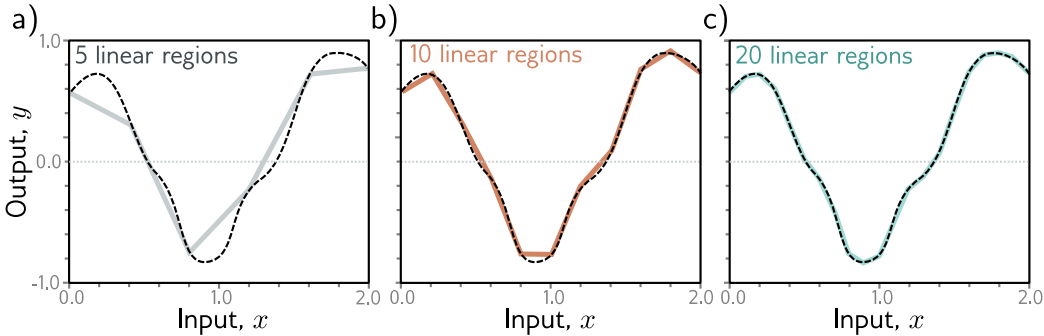
El argumento, completo (en una dimensión)

- Partimos $[0,1]$ en K tramos de ancho $1/K$ y conectamos los puntos $(x_k, f(x_k))$: eso usa del orden de K unidades ReLU.
- Dentro de cada tramo, $\hat{f}(x)$ es un promedio ponderado de $f(x_k)$ y $f(x_{k+1})$, así que

$$\sup_x |f(x) - \hat{f}(x)| \;\leq\; \omega_f\left(\tfrac{1}{K}\right), \qquad \omega_f(\delta) = \sup_{|x-x'|\leq\delta} |f(x) - f(x')|$$

- f continua en un compacto es uniformemente continua: $\omega_f(1/K) \to 0$ cuando $K \to \infty$. □
- Lo mismo que ya conocemos: **splines** lineales, o una regresión con muchas funciones base.
- El costo: en p dimensiones el número de "celdas" crece como K^p. La maldición de la dimensionalidad no desaparece.

Teorema de aproximación universal



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- Más unidades ocultas $\Rightarrow$ más regiones lineales $\Rightarrow$ mejor aproximación.

Representar no es aprender

- El teorema es de **existencia**: dice que hay parámetros que aproximan f.
- No dice:
  1. cuántas unidades hacen falta (pueden ser muchísimas);
  2. que el gradiente descendente vaya a encontrar esos parámetros (la pérdida no es convexa);
  3. que la función que encontramos generalice. Con suficientes unidades, la red puede pasar por **todos** los puntos de entrenamiento, ruido incluido.
- Un modelo que puede aproximar cualquier cosa también puede sobreajustar cualquier cosa. De ahí la segunda mitad de la clase: **regularización**.

Agenda

1 Recap

2 Teorema de aproximación universal

3 Redes profundas

4 Regularización

5 Cierre

Si una capa alcanza, ¿para qué más capas?

- Las redes modernas tienen muchas capas ocultas, cada una de tamaño moderado.
- En la práctica, varias capas de tamaño modesto se entrenan más fácil y generalizan mejor que una sola capa enorme.
- Y hay un argumento de eficiencia: con la misma cantidad de unidades, la profundidad produce **exponencialmente más** regiones lineales (Montúfar, Pascanu, Cho & Bengio, 2014).

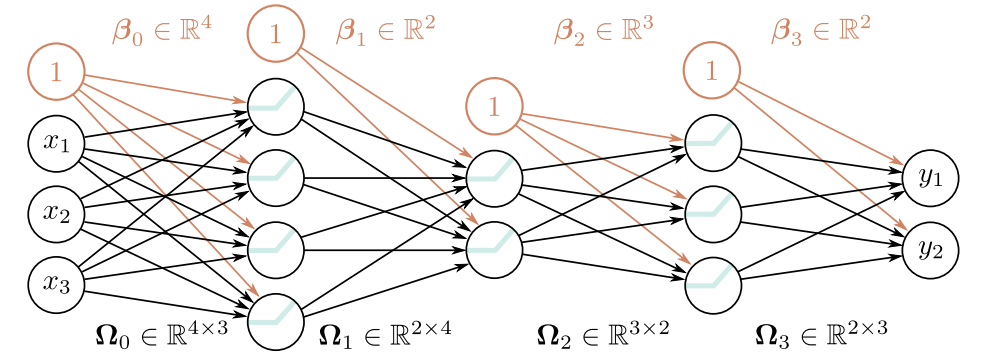
Multilayer Neural Networks

- Con K capas ocultas, cada capa toma la salida de la anterior:

$$
\begin{aligned}
h_1 &= g\left(\beta_0 + \Omega_0\, x\right) \\
h_k &= g\left(\beta_{k-1} + \Omega_{k-1}\, h_{k-1}\right), \qquad k = 2, \ldots, K \\
f(x) &= f^{(output)}\left(\beta_K + \Omega_K\, h_K\right)
\end{aligned}
$$

- Ω_k es la matriz de pesos y β_k el vector de interceptos (*biases*) de cada capa.
- Es una composición de funciones: lineal, no lineal, lineal, no lineal, ...

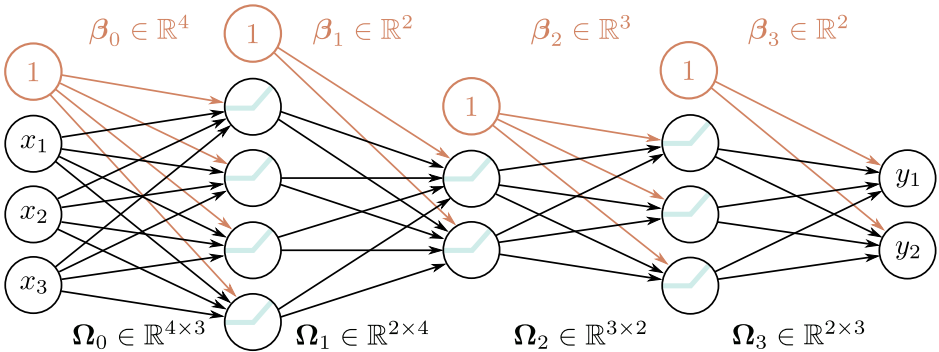
Multilayer Neural Networks



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- ¿Cuántos parámetros tiene esta red?

Multilayer Neural Networks



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- ¿Cuántos parámetros tiene esta red? $(12+4)+(8+2)+(6+3)+(6+2) = 43$

Arquitectura: ancho y profundidad

- La **arquitectura** es cuántas capas, cuántas unidades por capa y cómo se conectan.
- No hay teoría que diga cuál es la correcta para un problema dado.
- Ancho y profundidad son **hiperparámetros**: se eligen como siempre en este curso, mirando el error en un conjunto de validación.
- Y, de nuevo, una red más grande aproxima mejor, pero también sobreajusta más.

Agenda

1. Recap
2. Teorema de aproximación universal
3. Redes profundas
4. Regularización
5. Cierre

Network Tuning

- Entrenar una red requiere muchas decisiones que afectan el desempeño:
  - número de capas ocultas y de unidades por capa;
  - detalles del SGD: tasa de aprendizaje, tamaño del mini-batch, número de épocas;
  - parámetros de regularización.
- Las redes dada su flexibilidad tienden a sobreajustar
- Formas de regularizar:
  - **Explícita**: penalizar los pesos (L2, L1).
  - **Implícita** (heurísticas): early stopping, dropout.

Regularización explícita. Ridge

- á la Ridge

$$\min_{\Theta} \sum_{i=1}^{n} L\big(y_i, f(x_i;\Theta)\big) + \lambda \sum_j w_j^2$$

- Si la red no tiene capas ocultas, el output es la identidad y la pérdida es MSE, *es* Ridge.
- Con $\lambda \sum_j |w_j|$ tenemos la versión Lasso; con las dos, elastic net.
- Normalmente se penalizan los pesos y no los sesgos

¿Por qué se llama *weight decay*?

- Notemos que sucede en gradiente descendiente cuando tenemos la penalidad de Ridge:

$$w \leftarrow w - \epsilon \left(\nabla_w L + 2\lambda w \right)$$

¿Por qué se llama *weight decay*?

- Notemos que sucede en gradiente descendiente cuando tenemos la penalidad de Ridge:

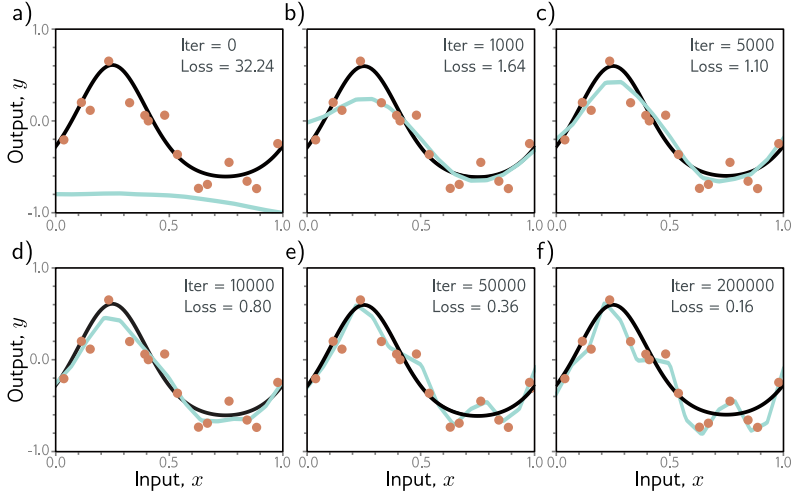
$$w \leftarrow w - \epsilon \left(\nabla_w L + 2\lambda w \right)$$

- Reordenando:

$$w \leftarrow \underbrace{\left(1 - 2\epsilon\lambda\right)}_{<1} w - \epsilon \nabla_w L$$

- En cada paso, antes de moverse en la dirección del gradiente, los pesos se **encogen** por un factor fijo.
- En cada paso se encojen hacia cero

Early Stopping

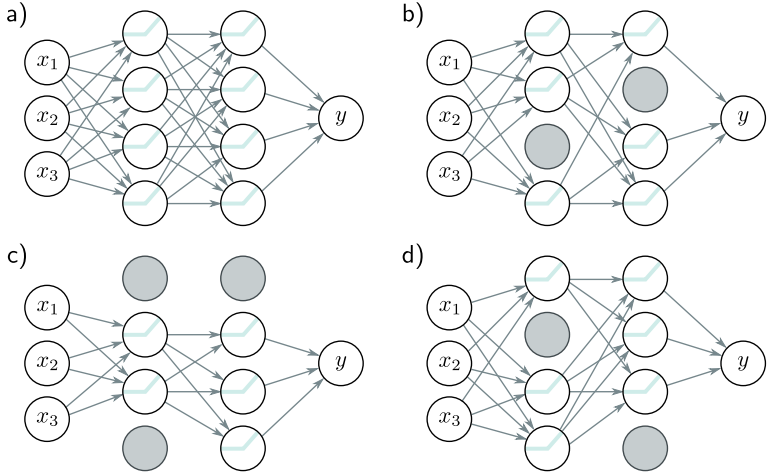


Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Early Stopping

- Al principio la red aprende la forma general; si seguimos iterando, aprende el ruido.
- **Early stopping**: monitorear la pérdida en validación y parar cuando deja de mejorar. Nos quedamos con los pesos de la mejor época.
- Limitar las iteraciones se comporta como una penalidad L2, con $\lambda \approx \frac{1}{\epsilon\tau}$ (Goodfellow, Bengio & Courville, 2016, §7.8).
- El número de épocas se vuelve un hiperparámetro más, y casi no hay que preocuparse

Dropout



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Dropout

- En cada mini-batch, cada unidad oculta se ”apaga” con probabilidad ϕ (Srivastava et al., 2014):

$$\tilde{h}_k = \frac{m_k}{1-\phi} h_k, \qquad m_k \sim \text{Bernoulli}(1-\phi)$$

- Dividir por $1-\phi$ mantiene el valor esperado: $\mathbb{E}[\tilde{h}_k] = h_k$.
- En predicción se usan todas las unidades.

Dropout

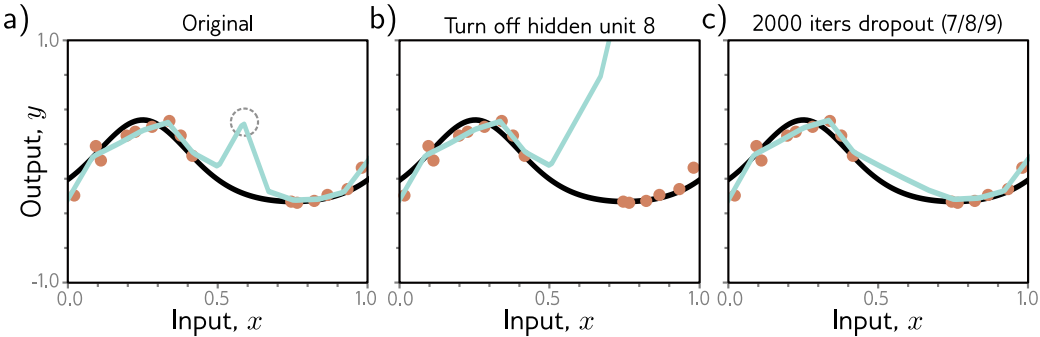
- En cada mini-batch, cada unidad oculta se ”apaga” con probabilidad ϕ (Srivastava et al., 2014):

$$\tilde{h}_k = \frac{m_k}{1-\phi} h_k, \qquad m_k \sim \text{Bernoulli}(1-\phi)$$

- Dividir por $1-\phi$ mantiene el valor esperado: $\mathbb{E}[\tilde{h}_k] = h_k$.
- En predicción se usan todas las unidades.

- ¿Por qué funciona?
  - Ninguna unidad puede depender de que otra unidad específica esté prendida: se evitan las ”co-adaptaciones”.
  - Es como promediar 2^K subredes que comparten pesos: la misma idea de **bagging** y de los random forests, que aleatorizan para promediar modelos distintos.

Dropout



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Batch Normalization

- Dentro de cada mini-batch B, se estandariza la entrada de cada unidad y después se reescala con parámetros que se aprenden:

$$\hat{z} = \frac{z - \mu_B}{\sigma_B}, \qquad \tilde{z} = \gamma \hat{z} + \delta$$

- Ioffe & Szegedy (2015). Su objetivo principal es **estabilizar y acelerar** el entrenamiento: cada capa recibe entradas en una escala parecida.
- Regulariza un poco como efecto secundario, porque μ_B y σ_B cambian de un mini-batch a otro y meten ruido.
- es complementario

Ejemplo: MNIST



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1. Recap
2. Teorema de aproximación universal
3. Redes profundas
4. Regularización
5. Cierre

Recap

1. Una red de una capa, suficientemente ancha, aproxima cualquier función continua.
2. Representar no es aprender: el teorema no dice nada sobre optimización ni sobre generalización.
3. Regularizar es clave para evitar el sobreajuste.